Sliding Window Pattern

DSA Pattern Recognition • Fixed Size & Variable Length

1. What is Sliding Window?

Sliding Window is mainly used when a problem asks about a **continuous or contiguous** part of an array or string. Instead of recalculating every subarray or substring from scratch, we move the window and maintain useful information.

```
Array:    2   1   5   1   3   2

Window:
              [ 1   5   1 ]
                ↑       ↑
               low     high
```

2. When should you think Sliding Window?

A. Contiguous / Continuous

Look for words such as: **subarray, substring, consecutive elements, continuous segment**.

B. Fixed Size

Examples: “Find the maximum sum of a subarray of size k” or “Find the average of every k consecutive elements.”
→ Think **Fixed Sliding Window**.

C. Variable Size + Condition

Examples: “Longest substring with at most K distinct characters”, “Smallest subarray whose sum is ≥ target”, or “Longest subarray satisfying a condition.”
→ Think **Variable-Length Sliding Window**.

3. Variable-Length Sliding Window — General Template

Use this when the window size is **not fixed**. Expand with **high** and shrink from **low** whenever the window becomes invalid.

```
int low = 0;
int ans = 0;

for (int high = 0; high < n; high++) {

    // 1. Add high element to the window
    // e.g. freq[arr[high]]++;

    // 2. Shrink while the window is invalid
    while (window is invalid) {

        // Remove low element
        // e.g. freq[arr[low]]--;

        low++;
    }

    // 3. Window is now valid
```

```
    int len = high - low + 1;

    // 4. Update answer
    ans = max(ans, len);
}
```

Key idea:

Expand → Check condition → Shrink until valid → Calculate answer → Continue.

4. Example: Longest Substring Without Repeating Characters

```
int low = 0;
int ans = 0;

for (int high = 0; high < s.length(); high++) {

    freq[s[high]]++;

    while (freq[s[high]] > 1) {
        freq[s[low]]--;
        low++;
    }

    ans = max(ans, high - low + 1);
}
```

Here, the window is invalid when a character appears more than once. We move **low** until the substring becomes valid again.

5. Fixed-Size Sliding Window — General Template

Use this when the problem explicitly gives a window size **k**.

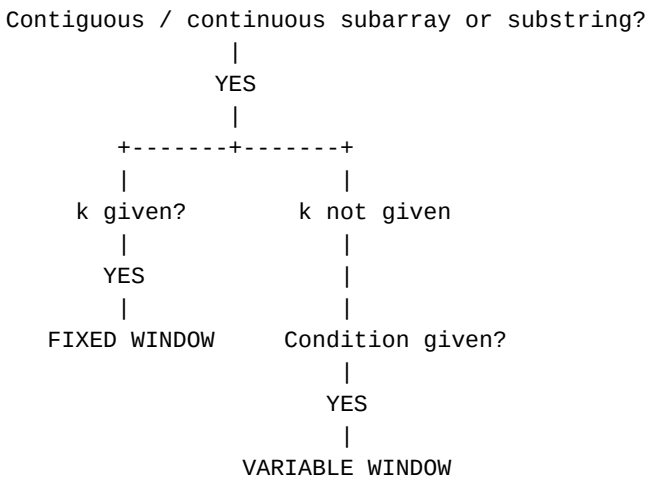
```
int low = 0;
int ans = INT_MIN;

for (int high = 0; high < n; high++) {

    // 1. Add high element
    // window += arr[high];

    // 2. Check whether window size is k
    if (high - low + 1 == k) {

        // 3. Calculate / update answer
        ans = max(ans, window);

        // 4. Remove low element
        // window -= arr[low];

        // 5. Move low
        low++;
    }
}
```

6. Complexity

Pattern	Time	Space	Main Condition

Fixed Sliding Window	O(n)	O(1) usually	window size == k
Variable Sliding Window	O(n)	O(k) / O(1)	shrink while invalid

7. Interview Recognition Cheat Sheet



Remember:

• **k is given** → Fixed Sliding Window → usually use **if (high - low + 1 == k)**.

• **k is not given + condition** → Variable Sliding Window → usually use **while (window is invalid)**.

8. Interview Answer

“I use the sliding window technique mainly when the problem involves a contiguous subarray or substring. If the window size is fixed, I maintain a fixed-size window. If the size is variable, I expand the right pointer and shrink the left pointer whenever the window violates the required condition. This avoids recalculating every subarray or substring and usually reduces the time complexity to O(n).”

One-line memory trick: Contiguous + moving range + optimize over subarrays/substrings → Think Sliding Window.